J Cryptogr Eng (2011) 1:265–270
DOI 10.1007/s13389-011-0022-y

REGULAR PAPER

An on-chip glitchy-clock generator for testing fault injection attacks

Sho Endo · Takeshi Sugawara · Naofumi Homma · Takafumi Aoki · Akashi Satoh

Received: 23 September 2011 / Accepted: 30 September 2011 / Published online: 21 October 2011

Abstract This paper presents a glitchy-clock generator integrated in FPGA for evaluating fault injection attacks and their countermeasures on cryptographic modules. The proposed generator exploits clock management capabilities, which are common in modern FPGAs, to generate clock signal with temporal voltage spike. The shape and timing of the glitchy-clock cycle are configurable at run time. The proposed generator can be embedded in a single FPGA without any external instrument (e.g., a pulse generator and a variable power supply). Such integration enables reliable and reproducible fault injection experiments. In this paper, we examine the characteristics of the proposed generator through experiments on Side-channel Attack Standard Evaluation Board (SASEBO). The result shows that the timing of the glitches can be controlled at the step of about 0.17 ns. We also demonstrate its application to the safe-error attack against an RSA processor.

Keywords Fault injection attacks · Clock glitch · RSA · Safe-error attack

1 Introduction

Fault injection attacks are attracting much attention in the field of cryptographic hardware and embedded systems. The attackers first inject faults to cryptographic operations, and then estimate a secret key from the several faulty ciphertexts.

Boneh et al. [1] first proposed Bell-core attack against RSA cryptosystems based on Chinese Reminder Theorem (CRT). The attack calculates the difference between correct and faulty (i.e., fault-injected) outputs, which makes it possible to factorize the modulus used in RSA encryption/decryption. Yen and Joye then proposed Safe-error attack against public-key cryptosystems with Multiply-and-squaring-always method which is a conventional countermeasure to timing attack (TA) and simple power analysis (SPA) [2]. The attack is undetectable by common recalculation schemes performed just before the output since the key information is revealed when the fault is injected into a dummy operation.

After the first publication focusing on public-key cryptosystems [1], the fault injection attacks were also extended to symmetric-key cryptosystems. Differential fault analysis (DFA) [3] extracts secret key from the difference of correct and faulty outputs. In DFA, we assume that temporal faults are injected into some bits only for a short time and the faulty values are propagated through the following hardware logics without any additional faults. Ineffective fault analysis (IFA) [4] injects faults into specific operation and observe secret key from whether the output changes or not. Such advanced attacks sometimes require timely injected faults during specific operations.

With the advance of fault injection attacks and countermeasures, fault injection techniques have also been investigated to evaluate the possibility of the attacks in practice. Fault models are roughly categorized into two groups, namely permanent and transient faults. In a permanent fault, a target circuit is damaged forever. It is rather easy to detect such faults and react to them by means of power-on self-test (POST). On the other hand, a transient fault is induced at run time and the target circuit can be recovered at the end of

S. Endo (✉) · T. Sugawara · N. Homma · T. Aoki
Tohoku University, 6-6-05, Aramaki Aza Aoba, Aoba-ku, Sendai 980-8579, Japan
e-mail: endo@aoki.ecei.tohoku.ac.jp

A. Satoh
National Institute of Advanced Industrial Science and Technology, 1-1-1 Umezono, Tsukuba, Ibaraki 305-8568, Japan

operation. For both fault models, various injection techniques were reported using glitches on power and clock signals, lower voltages, higher frequencies, laser shots, light illuminations on the surface of a depackaged chip, and so on [5–7]. Among them, a transient fault caused by a glitchy-clock signal (i.e., a clock signal with a glitch) is one of the possible faults due to the non-invasiveness and controllability.

This paper presents a glitchy-clock generator integrated in FPGA. (A preliminary short version of this paper was presented at COSADE 2011 [8].) The proposed method is based on the method by Fukunaga, et al. [9], where a temporal voltage spike is injected to clock by switching between two clock signals with the same frequency but slight phase differences. A glitchy-clock cycle occurs when the clock source is switched from one to another. The experimental environment in [9] requires external equipments such as a pulse generator and an oscilloscope and tune them manually to perform consistent experiments. In contrast, our major contributions of this paper are to (i) integrate the glitchy-clock generator into a single FPGA without any external pulse generator and (ii) add a further functionality to configure the shape and timing of the glitchy-clock signal at run time. In practice, the proposed generator is implemented in an FPGA on the Side-channel Attack Standard Evaluation Board (SASEBO) [10] and, therefore, it can be used as a common experimentation environment for fault injection attacks, where the glitchy-clock signals are reproducible. It can also be used to investigate sophisticated attacks combining fault injection attacks and side-channel attacks [11].

In this paper, we first evaluate the basic characteristics of the proposed generator implemented on SASEBO. The result shows that the glitch signal can be injected timely for any clock cycle (i.e., tick) in increments of about 0.17 ns. This paper also demonstrates the effectiveness of the proposed generator through the safe-error attack against RSA processor, where fault occurrences are examined using the corresponding power traces instead of outputs. The result shows that we can successfully distinguish between normal and dummy operations just by observing difference traces.

2 On-chip glitchy-clock generator

2.1 Concept of glitchy-clock generation

Figure 1 shows the concept of the glitchy-clock generator which creates a clock signal with a glitchy-clock cycle. The timing of the glitchy-clock cycle can be controlled to inject a transient fault into a specific operation or round. The parameters of the glitch are defined as Fig. 1, where T_w is the glitch width given as a period between the first rising and falling edges and T_p is the glitch period given as a period between the first and second rising edges.

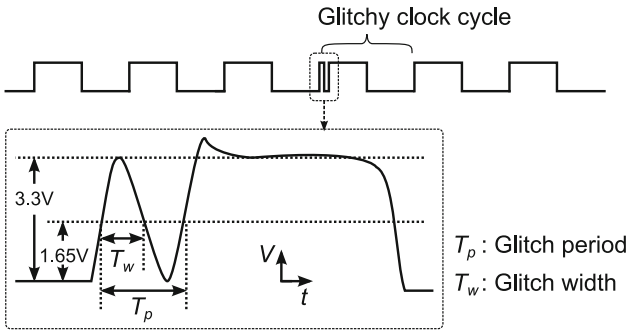

Fig. 1 Image of clock signal with glitchy-clock cycle

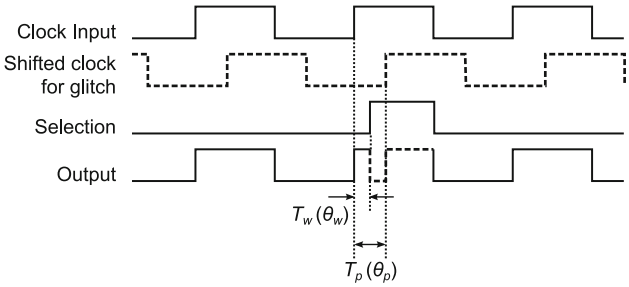

Fig. 2 Concept of glitch generation

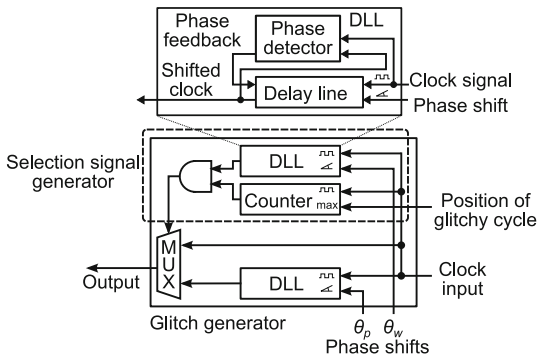

Fig. 3 Glitch generator

The proposed method considered here is to generate the above glitchy-clock signal using two clock signals with different phases. The glitchy-clock cycle occurs when the clock source is switched from one to another. In [9], such two signals are generated by an external equipment. In contrast, the proposed method generates them by an internal function on a chip. Figure 2 shows the concept of the glitchy-cycle generation. The two clock signals with different phases are fed into a multiplexer and switched at a specific timing. We can change T_p and T_w by the phases of "Shifted clock for glitch" signal and "Selection" signal, respectively.

Figure 3 shows a block diagram of the glitch generator. It consists of a counter and two delay locked loop (DLL) circuits in digital clock managers (DCMs) available on Xilinx FPGAs. In Fig. 3, selection signal is given by the counter output and the clock signal with a different phase. We can

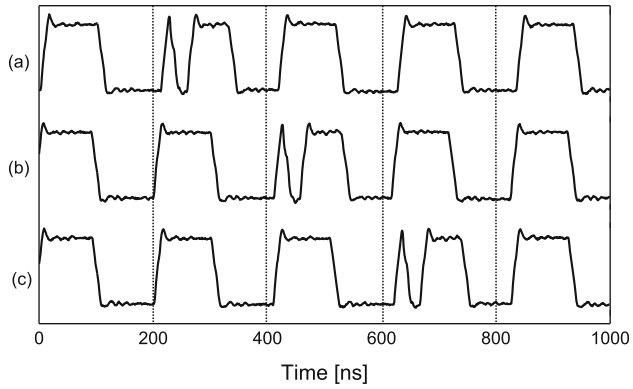

Fig. 4 Examples of generated glitchy-clock signals

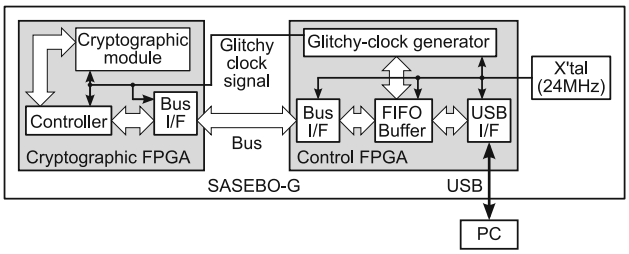

Fig. 5 Proposed fault injection system on SASEBO-G

program the DCMs and control the phase-shift parameters θ_p and θ_w. Note that such capability is common in modern FPGAs (e.g. PLL for Altera FPGAs), and thus it is easy to implement the proposed generator on FPGAs produced by other vendors.

Figure 4 also illustrates examples of generated clock signals with glitchy-clock cycles, where (a), (b), and (c) are the clock signals with glitches at the 1st, 2nd, and 3rd clock cycles, respectively. A glitchy-clock cycle can be induced into any cycle depending on the maximum counter value. In summary, the proposed generator has the following functions:

- Change the period and width of the glitch within one clock cycle.
- Induce a glitchy-clock cycle (i.e., a clock tick with a glitch) into any cycle.

Figure 5 shows a block diagram of fault injection system implemented on Side-channel Attack Standard Evaluation Board with two Xilinx FPGAs (SASEBO-G). The proposed generator is implemented in one FPGA (VirtexII-Pro XC2VP30). The output of the proposed generator is fed to the target chip (VirtexII-Pro XC2VP7). Phase-shift parameters for DCMs can be controlled from an external PC through a series of communication interfaces (a FIFO and an USB I/F).

Basic characteristics of the proposed glitch generator are evaluated on SASEBO-G. We focus on the resolution of the parameter T_p while the parameter T_w is fixed. Several clock signals are measured using a digital oscilloscope while changing the glitch periods from 4.9 to 17.9 ns. Note that the glitch width is fixed to 4.2 ns. Figure 6 shows the overlapped traces corresponding to the different glitch widths. The magnified view in Fig. 6 shows that we can tune the glitch width precisely in increments of about 0.17 ns. More precisely, the increment size follows the normal distribution $N(\mu, \sigma^2) = N(0.17, 0.0013)$, which corresponds to the minimum amount of phase shift in DCM. Note that the resolution of T_d is especially important to some advanced attacks, such as Fault Sensitivity Attack [12], which exploits data-dependent critical path variations.

2.2 Glitch generation in low-frequency clock

The output frequency of the proposed generator is limited to that of DLL in FPGA. When the clock frequency is lower than the minimum frequency, the generator described above cannot generate glitchy-clock signals directly. In that case, the generator needs to divide the output frequency by n.

Figure 7 illustrates a timing chart of the proposed generator with frequency dividing. The original clock signal is divided by 2, and the obtained clock signal (i.e., Divided clock) is fed into the following circuit or system. Thus, we can generate glichty-clock signals in lower frequency. Figure 8 shows a block diagram of the modified generator, where a divider is inserted to generate Divided clock.

Figure 9 shows the overlapped traces corresponding to the different glitch widths in the modified generator, where the glitch width is changed from 0.0 to 17.5 ns. Note here that the configurable glitch width of the modified generator is reduced by half as compared with that of the above generator. When the clock frequency is divided by n, the configurable

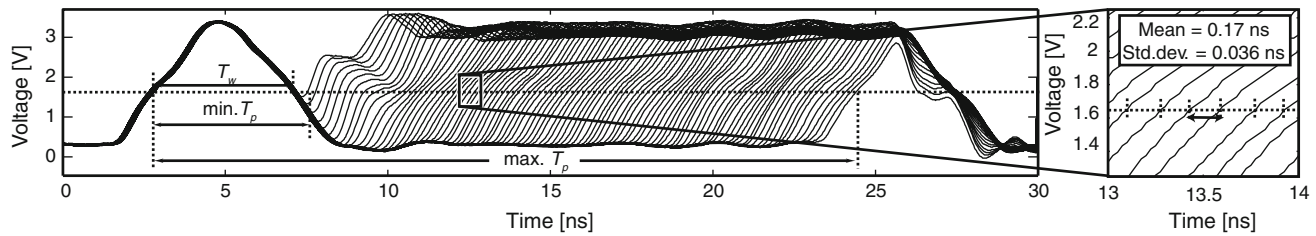

Fig. 6 Waveforms of glitchy-clock cycles for different glitch widths

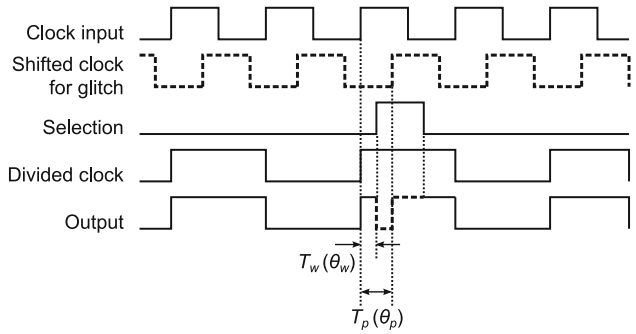

Fig. 7 Glitch generation with frequency dividing

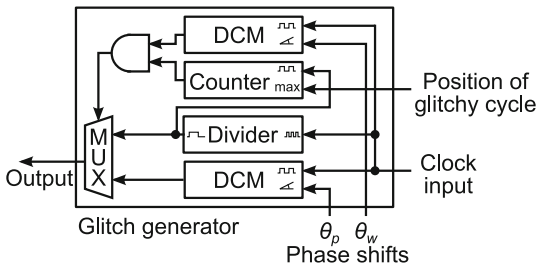

Fig. 8 Glitch generator with frequency divider

range is reduced to one nth due to the width of the shifted clock.

Table 1 summarizes the minimum output frequencies of FPGAs mounted on the SASEBO series. The modified generator is required if a fault injection experiment is performed by a lower clock frequency on a SASEBO.

Table 1 Minimum output frequencies

SASEBO type	FPGA	Minimum output frequency (MHz)
SASEBO-G	Xilinx XC2VP30	24
SASEBO-R	Xilinx XC2VP30	24
SASEBO-B	Altera EP2S30	4.6875
SASEBO-GII	Xilinx XC3S400A	5
SASEBO-G	Xilinx XC6SLX150	5

ALGORITHM 1
SQUARE- AND- MULTIPLY ALWAYS METHOD

Input:	$X, N,$ $E = (e_{k-1}, ..., e_1, e_0)_2$
Output:	$X^E \bmod N$

1 : $R := 1;$
2 : **for** $i = k - 1$ **downto** 0 **do**
3 : $R_0 := R \cdot R \bmod N;$ — squaring
4 : $R_1 := R_0 \cdot X \bmod N;$ — multiplication
5 : $R := R_{e_i}$
6 : **end for**
7 : **return** R

3 Application to safe-error attack on RSA

3.1 Safe-error attack

Safe-error attack [2] is a fault injection attack on a classical modular exponentiation algorithm called the squaring-and-multiply always method [13] shown in Algorithm 1. The exponentiation method is a basic countermeasure against SPA which inserts dummy multiplications to the left-to-right binary method [14]. The algorithm prevents an attacker from finding the different trace patterns between multiply and squaring operations depending on a secret exponent. In the safe-error attack, an attacker injects a carefully synchronized fault during the multiplication process. If the fault-injected multiplication was dummy, the faulty intermediate value never propagates to succeeding operations because it is discarded. Therefore, the attacker can distinguish normal and dummy multiplications by checking the outputs. As a result, the attacker gains one bit of the secret exponent in each injection and recover the whole secret exponent after several iterations.

The attack considered here is an extension of the safe-error attack which uses power traces instead of output to distinguish dummy operations from others. Here, the attacker captures a pair of power traces: ones with and without fault injection to a multiplication operation. Then, the attacker examines the difference between the traces. If it was a dummy multiplication, then the following operations are identical, thus a difference between the traces should be small. In other case where the multiplication was not a dummy, the following operations after the injection are faulty, and then a larger

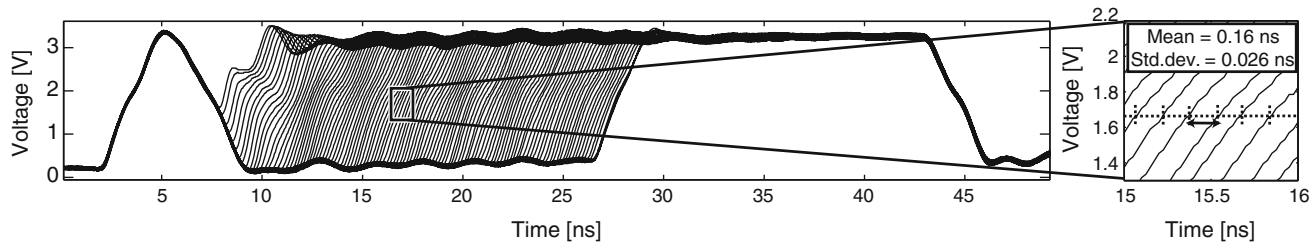

Fig. 9 Waveforms of glitchy-clock cycles for low frequency clock

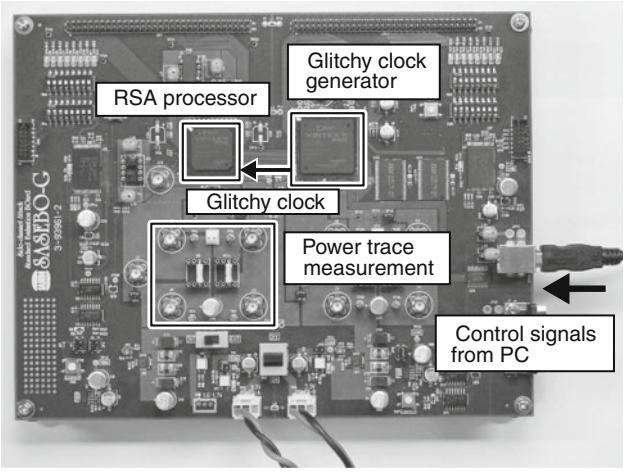

Fig. 10 Experimental setup

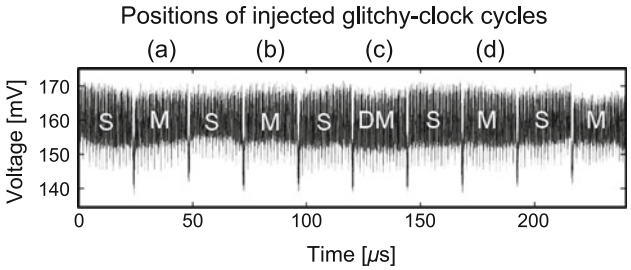

Fig. 11 Power trace of RSA processor

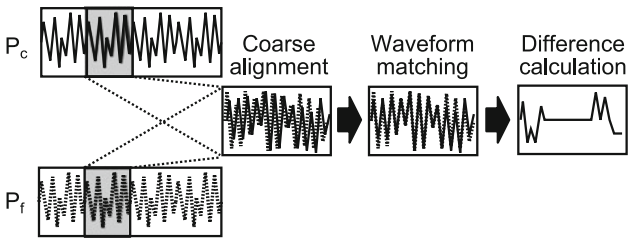

Fig. 12 Difference calculation with waveform matching

difference should appear. As a result, the attacker can distinguish dummy multiplications from others just by observing the difference trace.

3.2 Target processor

An RSA processor with high-radix Montgomery multiplier [15] is used in this experiment. The datapath includes a multiplication block, which repeats the multiply-additions in accordance with the squaring-and-multiply always method with a 512-bit exponent. The 32-bit datapath performs multiply-additions using 32-bit operands stored in the registers. Each multiplication and squaring takes 578 cycles.

We examined the critical path of the target circuit by sweeping the glitch width using the proposed generator. The error rate is measured by changing the glitch periods T_p from 6.0 to 13.5 ns, where the glitch delay is fixed to 4.9 ns. 100 fault injection tests are performed for each glitch width. As a result, we obtained the error rate of 1.0 (i.e., 100% error) in the range $6.5 \leq T_p \leq 13.5$ ns. The shorter width did not succeed in generating the significant voltage drop before the 2nd positive edge arose. The wider width did not disturb any operation due to the operation margin. In the following experiment, we employed the glitch period of 9.7 ns to inject faults with high reliability.

3.3 Experiment

Figure 10 shows the experimental setup consisting of a SASEBO-G, an oscilloscope, and a PC. In the experiment, a pair of traces with and without fault injection is captured and the difference is examined as described in the Sect. 3.1. Note that a clock glitch is injected in synchronization with the target multiplication and the oscilloscope is triggered at the moment.

Figure 11 shows a measured power trace obtained from the RSA processor. The labels S, M, and DM indicate the squaring, multiplication and dummy multiplication operations, respectively.

The above attack observes the difference between two waveforms. Therefore, a precise matching technique which can overcome the cumulative effect of clock jitter and noise is crucial for the observation. In this experiment, a phase-based waveform matching technique is used [16], shown in Fig. 12, in which waveform positions can be matched with a resolution higher than the sampling resolution. The waveform segments are aligned precisely by the phase-based waveform matching technique, and then the difference between the waveforms is calculated to evaluate the equality of the operations.

Figure 13 shows differential power traces between the pairs of traces (ones with and without fault). Figure 13a–d corresponds to the results of fault injections on 1st–4th multiplications. Note that only the third one is a dummy multiplication. We can see a significant difference on the fault-injected multiplications in Fig. 13a–d. Differences after the injections are rather minor, but clearly show differences between normal and dummy multiplications. In case of dummy multiplication in Fig. 13c, the amplitude of the differential trace after the injection is smaller compared with those in Fig. 13a, b, and d. Therefore, the attacker can distinguish the traces and recognize that the 3rd exponent bit is 0. In contrast, the differential traces in Fig. 13a, b and d indicate that the original and faulty traces do not match. This means that the target operations are real multiplication operations, and the 1st, 2nd and 4th key bits are revealed to be 1. As a result, the attacker can

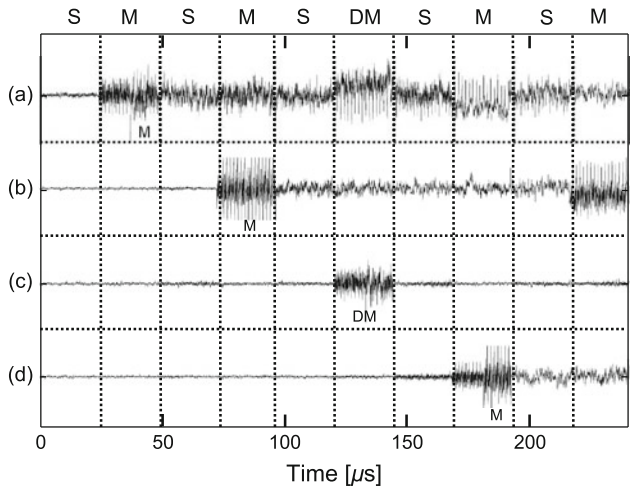

Fig. 13 Differential power traces

obtain the first four key bits $E = (1101)_2$ from the safe-error attack.

4 Conclusion

This paper presented an on-chip glitchy-clock generator for evaluating fault injection attacks and their related countermeasures. The proposed generator can be implemented in an FPGA on SASEBO without using any external equipment, and thus is suitable for a common evaluation environment to achieve reproducible experiments. The result shows that the glitches can be injected timely to any clock cycle in increments of about 0.17 ns. We also demonstrated its application to the safe-error attack against RSA processor. We confirmed that the secret exponent bits were successfully obtained by faults provided by the proposed generator. Further experiments are being conducted to apply it to sophisticated attacks such as fault sensitivity analysis [12].

References

1. Boneh, D., Demillio,R., Liotin, R.: On the importance of checking crypto-graphic protocols for fault. In: EUROCRYPT 1997, LNCS, vol. 1233, pp. 37–51. Springer, Berlin (1997)
2. Yen, S.M., Joye, M.: Checking before output may not be enough against fault-based cryptanalysis. IEEE Trans. Comput. **49**(9), 967–970 (2000)
3. Biham, E., Shamir, A.: Differential fault analysis of secret key cryptosystems. CRYPTO **1294**, 513–525 (1997)
4. Clavier, C.: Secret external encodings do not prevent transient fault analysis. LNCS **4727**, 181–194 (2007)
5. Bar-El, H., Choukri, H., Naccache, D., Tunstall, M., Whelan, C.: The sorcerer's apprentice guide to fault attack. IACR ePrint archive, vol. Report 2004/100, pp. 1–13 (2004)
6. Kim, C.H., Quisquater, J.-J.: Faults, injection methods, and fault attacks. IEEE Design Test Comput. **24**, 544–545 (2007)
7. Guilley, S., Sauvage, L., Danger, J.-L., Selmane, N., Pacalet, R.: Silicon-level solutions to counteract passive and active attacks. In: Proceedings of the 5th Workshop on Fault Diagnosis and Tolerance in Cryptography, pp. 3–17 (2008)
8. Endo, S., Sugawara, T., Homma, N., Aoki, T.: An on-chip glitchy-clock generator and its application to safe-error attack. In: 2nd International Workshop on Constructive Side-channel Analysis and Secure Design–COSADE, pp. 175–182 (2011)
9. Fukunaga, T., Takahashi, J.: Practical fault attack on a cryptographic lsi with iso/iec 18033-3 block ciphers. In: Proceedings of the 6th Workshop on Fault Diagnosis and Tolerance in Cryptography, pp. 84–92 (2009)
10. Side-channel Attack Standard Evaluation Board. http://www.rcis.aist.go.jp/special/SASEBO/
11. Amiel, F., Villegas, K., Feix, B., Marcel, L.: Passive and active combined attacks: combining fault attacks and side channel analysis. In: Proceedings of the 4th Workshop on Fault Diagnosis and Tolerance in Cryptography, pp. 92–102 (2007)
12. Li, Y., Sakiyama, K., Gomisawa, S., Fukunaga, T., Takahashi, J., Ohta, K.: Fault Sensitivity Analysis. Workshop on Cryptographic Hardware and Embedded Systems-CHES. LNCS **6225**, 320–334 (2010)
13. Coron, J.S.: Resistance against differential power analysis for elliptic curve cryptosystems. In: CHES 1999, LNCS, vol. 1717, pp. 292–302. Springer, Berlin (1999)
14. Menezes, J.A., Oorschot, C.P., Vanstone, A.S.: Handbook of Applied Cryptography. CRC Press, Boca Raton (1997)
15. Miyamoto, A., Homma, N., Aoki, T., Satoh, A.: Systematic design of high-radix montgomery multipliers for rsa processors. In: Proceedings of the 26th IEEE International Conference on Computer Design, pp. 416–422 (2008)
16. Homma, N., Miyamoto, A., Aoki, T., Satoh, A., Shamir, A.: Comparative power analysis of modular exponentiation algorithms. IEEE Trans. Comput. **59**(6), 795–807 (2010)